
```
#!/usr/bin/python3

# Run me like this:
# $ python3 padding_oracle.py
# "https://project1.eecs388.org/username/paddingoracle/verify" "5a7793d3..."

import json
import sys
import time
from typing import Union, Dict, List
from Crypto.Cipher import AES
from Crypto.Hash import HMAC, SHA256
from Crypto.Random import get_random_bytes
import requests

# Create one session for each oracle request to share. This allows the
# underlying connection to be re-used, which speeds up subsequent requests!

s = requests.session()

def oracle(url: str, messages: List[bytes]) -> List[Dict[str, str]]:
    while True:
        try:
            r = s.post(url, data={"message": [m.hex() for m in messages]})
            r.raise_for_status()
            return r.json()
            # Under heavy server load, your request might time out. If this happens,
            # the function will automatically retry in 10 seconds for you.
        except requests.exceptions.RequestException as e:
            sys.stderr.write(str(e))
            sys.stderr.write("\nRetrying in 10 seconds...\n")
            time.sleep(10)
            continue
        except json.JSONDecodeError as e:
            sys.stderr.write("It's possible that the oracle server is overloaded
right now, or that provided URL is wrong.\n")
            sys.stderr.write("If this keeps happening, check the URL. Perhaps your
username is not set.\n")
            sys.stderr.write("Retrying in 10 seconds...\n\n")
            time.sleep(10)
            continue

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print(f"usage: {sys.argv[0]} ORACLE_URL CIPHERTEXT_HEX", file=sys.stderr)
        sys.exit(-1)
    oracle_url, message = sys.argv[1], bytes.fromhex(sys.argv[2])

    if oracle(oracle_url, [message])[0]["status"] != "valid":
        print("Message invalid", file=sys.stderr)

    cipherfake=[0] * AES.block_size
    plaintext = [0] * AES.block_size
    current = 0
    byteArr = []
    number_of_blocks = int(len(message)/AES.block_size)
    blocks = [[]] * number_of_blocks
    for i in (range(number_of_blocks)):
```

```

        blocks[i] = message[i * AES.block_size: (i + 1) * AES.block_size]

    for z in range(len(blocks)-1):
        for i in range (1,17):
            for v in range(256):
                cipherfake[-i]=v
                bruh = oracle(oracle_url, [bytes(cipherfake)+blocks[z+1]])[0]

["status"]
                if bruh != "invalid_padding":
                    current = i
                    d = v^i
                    plaintext[-i]= d^blocks[z][-i]

            for w in range(1,current+1):
                cipherfake[-w] = plaintext[-w]^i+1^blocks[z][-w]

        for i in range(16):
            byteArr.append(plaintext[i])
    # print(byteArr)
    n = byteArr[-1]
    byteArr = byteArr[:-n]
    # print(byteArr)
    # print(byteArr[:-SHA256.digest_size])
    # fin = []
    # for b in byteArr:
    #     if b > 31 and b < 127:
    #         fin.append(b)
    # msg = bytearray(fin).decode("utf-8")
    # print(msg)
    print(bytearray(byteArr[:-SHA256.digest_size]).decode("utf-8"))

```